

意思決定コンソール



Thought for 2m 50s >

結論から言うと、

- 意思決定コンソールは**2枚目のスクショの方向性（ポートフォリオビュー）で正解**です。
- ただし「マーケティング意思決定」としては、指標定義とチャートの意味をもう一段はっきりさせた方がよいです。
- 旧版（1枚目の「今週のベストシナリオ Δ¥」など）は**廃止 or サブタブ扱い**でよく、メインは「ポートフォリオダッシュボード」に寄せるのをおすすめします。

1. まず整理：この画面でやりたいこと

この画面の役割はこう決めた方がブレません：

「過去 X 日間で実行されたマーケティング施策ポートフォリオ全体について、インクリメンタル収益（Δ¥）、リスク、モデル品質を一発で把握し、どの施策を Go/Canary/Hold にするか意思決定するためのコンソール」

- モデルの妥当性を掘るのは **Diagnostics** が担当
- 個別施策の因果設計をするのは **Causal Design / Policy Lab** が担当
- このコンソールは「**横断 + サマリ**」だけに集中させる

なので、今の構成：

- 上部 KPI 4つ
- 期間フィルタ（Last 14/28/56 days）
- Trend / Segment / Channel チャート
- Decision Cards テーブル

は方向性として合っています 🍌

ここから「マーケティングで意味が明快になるように」定義を固め直すのが次のステップです。

2. マーケ用 Decision Console の最終形（要件）

Claude に渡す用に、UI と指標定義をできるだけ誤解なしで書きます。

2.1 画面名・スコープ

- 画面タイトル：
Decision Console - マーケティング施策ポートフォリオ
- 対象データ：
 - `policy_decisions` 的なテーブル（例）
 - `policy_id`
 - `policy_name`
 - `dataset_id`
 - `segment`（任意）
 - `channel`（任意）
 - `decision` ∈ { `go`, `canary`, `hold` }
 - `delta_yen`（その施策の期待インクリメンタル収益）
 - `roi`（任意。あれば使用、なければ空）
 - `cas_score`（0-1 スケール）
 - `risk_score`（0-1 or 0-100 スケール）
 - `period_start`, `period_end` or `decision_date`
 - フィルタで選択された期間・条件に含まれる「**Go or Canary の施策**」を基本集計対象とする（Hold は一覧には出すが KPI からは除外）。

2.2 フィルタ領域

右上のバッジ+必要に応じてドロップダウン：

- 期間プリセット：
 - `Last 14 days`, `Last 28 days`, `Last 56 days`

- 将来拡張（今は UI だけ or ドロップダウンだけ仕込んでおくでも可）：
 - Dataset フィルタ
 - Channel フィルタ
 - Segment フィルタ

※ フィルタが変わると 全ての KPI / チャート / テーブルが一括で再計算 される。

2.3 KPI カード（上段4つ）

1. Total Incremental Profit

- ラベル：Total Incremental Profit
- 値：

$$\sum \text{delta_yen} \quad (\text{期間内 \& decision} \in \{\text{go, canary}\})$$
- サブラベル：選択期間のポートフォリオ全体のΔ¥合計

2. Average Δ¥ per Policy

- ラベル：Average Δ¥ / Policy
- 値：

$$\text{平均}(\text{delta_yen}) \quad (\text{同じく Go/Canary 施策})$$
- サブラベル：1施策あたりの平均インクリメンタル収益

3. Mean CAS

- ラベル：Mean CAS
- 値：

$$\text{平均}(\text{cas_score}) \quad (\text{Go/Canary})$$
- サブラベル：Target ≥ 0.75 （しきい値は設定ファイルから読み込めるとベター）

4. Portfolio CVaR (worst 10%)

- ラベル：CVaR (worst 10%)
- 値：
 - delta_yen の分布を施策単位で並べ、下位 10% の平均。

$$\text{cvar_10} = \text{mean}(\text{bottom_10_percent}(\text{delta_yen}))$$
- サブラベル：最悪10%シナリオの平均Δ¥（小さいほど安全）

※ ROI を上段に出したい場合は

「Total Incremental Profit」の隣に

$$\text{Portfolio ROI} = \sum \text{delta_yen} / \sum \text{cost}$$
 を置く案もありますが、
今は cost が安定していないので一旦保留でも OK。

2.4 中段チャート

① Δ¥ Trend

- 目的：期間内のポートフォリオ Δ¥ 推移を週単位で見る
- X 軸：week（ISO 週。例: 2025-W48）
- Y 軸： $\sum \text{delta_yen}$ （各週の Go/Canary 施策の合計）
- ビジュアル：週棒グラフ

② Segment Portfolio

- 目的：「どのセグメントがポートフォリオのΔ¥を稼いでいるか」を可視化
- 前提：
 - segment カラムが存在しない場合は **チャートごと非表示**（空の「Unknown」を出さない）
- 集計：
 - 粒度：segment
 - X 軸： $\sum \text{delta_yen}$ （セグメント別 Δ¥ 合計）
 - Y 軸：Mean CAS（セグメント内施策の平均 CAS）
 - 点サイズ：施策数 or $\sum \text{cost}$ （あれば）
- 直感：
 - 右上（Δ¥大きい & CAS高い）セグメントが「エース」
 - CAS のしきい値（例 0.75）で水平ラインを入れてもよい

③ Channel Performance

- 目的：チャネル別の貢献と効率を見る
- 前提：
 - `channel` カラムが存在しない場合はこのパネルを非表示
- 集計：
 - 粒度： `channel`
 - 左軸： Σ `delta_yen` (棒)
 - 右軸： `Mean ROI` (折れ線)
 - ROI がなければ、折れ線は非表示で `delta_yen` のみ表示
- ラベル：
 - X 軸：チャネル名
 - Y1： `Total Δ¥`
 - Y2： `ROI`

2.5 下段テーブル「Decision Cards」

- 役割：意思決定の最小単位（施策）を一覧する
- カラム：
 - `Policy`
 - `Dataset`
 - `Channel` (なければ `-`)
 - `Segment` (なければ `-`)
 - `Δ¥` (円表記)
 - `ROI` (% or ratio)
 - `CAS`
 - `Risk` (0-1 or 0-100)
 - `Verdict` (Go / Canary / Hold)
 - `Period` (例: 2025-11-18 ~ 2025-11-24)
- 上部フィルタ：
 - トグルボタン： `All / Go / Canary / Hold`
 - テキスト検索： `Search by policy name / dataset`
- 行クリック時：
 - `Causal Design` or `Diagnostics` の該当分析画面へのリンク

3. Claude 向け実装メモ + サンプルスクリプト

Claude にそのまま渡せるよう、実装目線で落とします。

3.1 バックエンド集計エンドポイント (要件)

```
GET /api/v1/decision-console/summary
```

クエリパラメータ：

- `from` (ISO date, required)
- `to` (ISO date, required)
- `dataset_id` (optional)
- `channel` (optional)
- `segment` (optional)

レスポンス JSON 構造：

```
{
  "kpis": {
    "total_delta_yen": number,
    "avg_delta_yen": number,
    "mean_cas": number,
    "cvar_10": number
  },
}
```

```

"trend": [
  { "week": "2025-W48", "delta_yen": 2264.0 },
  ...
],
"segments": [
  { "segment": "High-Value Urban", "total_delta_yen": 1200.0, "mean_cas": 0.82, "policy_count": 3 },
  ...
],
"channels": [
  { "channel": "email", "total_delta_yen": 1500.0, "mean_roi": 3.5 },
  ...
],
"decisions": [
  {
    "policy_id": "pol-001",
    "policy_name": "Email Campaign Q4",
    "dataset": "marketing_campaign_10k_processed",
    "channel": "email",
    "segment": "High-Value",
    "delta_yen": 133.0,
    "roi": 2.8,
    "cas": 0.87,
    "risk": 0.15,
    "verdict": "go",
    "period": "2025-11-18/2025-11-25"
  },
  ...
]
}

```

3.2 集計の Python サンプル (疑似コード)

```

from dataclasses import dataclass
from datetime import date
from typing import List, Optional, Dict, Any
import numpy as np
import pandas as pd

@dataclass
class DecisionRecord:
    policy_id: str
    policy_name: str
    dataset: str
    channel: Optional[str]
    segment: Optional[str]
    delta_yen: float
    roi: Optional[float]
    cas: float
    risk: float
    verdict: str # "go" | "canary" | "hold"
    decision_date: date

def compute_decision_console_summary(
    records: List[DecisionRecord],
    date_from: date,
    date_to: date,
) -> Dict[str, Any]:
    """Decision Console 用の集計を行う。

    - 期間内の Go/Canary 施策だけを KPI/チャートの対象にする
    - Hold も含めた全施策は decisions テーブルとして返す
    """
    df = pd.DataFrame([r.__dict__ for r in records])
    df = df[(df["decision_date"] >= date_from) & (df["decision_date"] <= date_to)]

    # KPI 集計対象: Go/Canary のみ
    kpi_df = df[df["verdict"].isin(["go", "canary"])]

```

```

if kpi_df.empty:
    # 空の場合はゼロ埋めて返す
    return {
        "kpis": {
            "total_delta_yen": 0.0,
            "avg_delta_yen": 0.0,
            "mean_cas": 0.0,
            "cvar_10": 0.0,
        },
        "trend": [],
        "segments": [],
        "channels": [],
        "decisions": df.to_dict(orient="records"),
    }

total_delta_yen = float(kpi_df["delta_yen"].sum())
avg_delta_yen = float(kpi_df["delta_yen"].mean())
mean_cas = float(kpi_df["cas"].mean())

# CVaR (worst 10%)
sorted_delta = np.sort(kpi_df["delta_yen"].values)
cutoff = max(1, int(len(sorted_delta) * 0.1))
cvar_10 = float(sorted_delta[cutoff].mean())

# 週次 Δ trend
week_df = kpi_df.copy()
week_df["week"] = week_df["decision_date"].dt.isocalendar().week.astype(str)
trend = (
    week_df.groupby("week")["delta_yen"]
    .sum()
    .reset_index()
    .to_dict(orient="records")
)

# セグメントポートフォリオ (segment が存在する場合のみ)
segments = []
if "segment" in kpi_df.columns and kpi_df["segment"].notna().any():
    segments = (
        kpi_df.dropna(subset=["segment"])
        .groupby("segment")
        .agg(
            total_delta_yen=("delta_yen", "sum"),
            mean_cas=("cas", "mean"),
            policy_count=("policy_id", "nunique"),
        )
        .reset_index()
        .to_dict(orient="records")
    )

# チャネルパフォーマンス (channel が存在する場合のみ)
channels = []
if "channel" in kpi_df.columns and kpi_df["channel"].notna().any():
    def mean_ignore_nan(x):
        x = x.dropna()
        return x.mean() if len(x) > 0 else np.nan

    channels = (
        kpi_df.dropna(subset=["channel"])
        .groupby("channel")
        .agg(
            total_delta_yen=("delta_yen", "sum"),
            mean_roi=("roi", mean_ignore_nan),
        )
        .reset_index()
        .to_dict(orient="records")
    )

return {
    "kpis": {

```

```

        "total_delta_yen": total_delta_yen,
        "avg_delta_yen": avg_delta_yen,
        "mean_cas": mean_cas,
        "cvar_10": cvar_10,
    },
    "trend": trend,
    "segments": segments,
    "channels": channels,
    "decisions": df.to_dict(orient="records"),
}

```

What this does (上のコードの要約)

- 入力：期間内の施策決定レコード（policy・ $\Delta\%$ ・CAS・チャネル・セグメントなど）。
- 出力：Decision Console 画面に必要な JSON（KPI / 週次トレンド / セグメント / チャネル / 一覧）。
- Go/Canary のみを KPI・チャート対象にしつつ、全レコードはテーブルに返す。`segment` や `channel` が無いときは、そのチャートを自動的に非表示にできるよう、空配列を返している。

Expert insight (Google/Meta/NASA level)

****ポートフォリオレビューの最重要ポイントは「指標の一貫性」**です。**
 KPI・チャート・テーブルで使っている $\Delta\%$ ・CAS・CVaR の定義が1か所でもズレると、
 利用者は「数字は合っているのか？」を疑い始め、ダッシュボード全体の信頼性が落ちます。
 実装時は「**単一の集計エンドポイントを真実のソース (SSOT)**」にするのが鉄則です。

今のスクショの UI はかなり良いところまで来ているので、

- 上記のように「指標の定義」と「チャネル/セグメントが無いときの挙動」だけきっちり仕様化
 - 旧 v1 画面は `decision-console-v1` のような別タブに「Legacy / Sandbox」として残すか、完全に削除
- くらいの整理で、「マーケの意思決定コンソール」として十分に世界レベルを名乗れると思います。